VoiceMe MongoDB Document Schemas

With MongoDB, you can embed data directly inside collections to keep things simple, fast, and schema-flexible. Instead of relational tables, here are the three core collections using Mongoose (Node.js) schemas.

1. `User` Schema (`users` collection)

Handles all three user types (**autistic_user**, **parent**, **caregiver**). Guardians hold references to the autistic individuals they manage.

JavaScript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  fullName: { type: String, required: true },
  email: { type: String, unique: true, sparse: true }, // Optional for end-users
  password: { type: String }, // Hashed password
  role: {
    type: String,
    enum: ['autistic_user', 'parent', 'caregiver'],
    required: true
  },
  accessibilitySettings: {
    ttsVoiceId: { type: String, default: 'en-US-Standard-A' },
    speechSpeed: { type: Number, default: 1.0 },
    highContrast: { type: Boolean, default: false }
  },
  // If role is 'parent' or 'caregiver', list the IDs of autistic users they manage
  managedUsers: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }],
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('User', userSchema);
```

2. `Flashcard` Schema (`flashcards` collection)

Stores visual communication cards assigned directly to an autistic user.

JavaScript

```
const flashcardSchema = new mongoose.Schema({
  // The autistic user this flashcard belongs to
  autisticUserId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  // Who created it (Parent, Caregiver, or Self)
  createdById: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  label: { type: String, required: true },       // e.g., "I need a break"
  spokenText: { type: String, required: true },  // e.g., "I am overwhelmed and need a quiet
space."
  imageUrl: { type: String, required: true },    // Cloud storage image link
  isEmergency: { type: Boolean, default: false }, // Pin to top SOS cards
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Flashcard', flashcardSchema);
```

3. `Routine` Schema (`routines` collection)

Instead of a separate junction table, MongoDB allows embedding the ordered list of flashcard steps directly inside the routine document.

JavaScript

```
const routineSchema = new mongoose.Schema({
  autisticUserId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  title: { type: String, required: true }, // e.g., "Morning School Prep"
  // Embedded array of steps (No separate table needed)
  steps: [{
    flashcardId: { type: mongoose.Schema.Types.ObjectId, ref: 'Flashcard' },
    stepOrder: { type: Number, required: true },
    isCompleted: { type: Boolean, default: false }
  }],
  isActive: { type: Boolean, default: true },
```

```
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Routine', routineSchema);
```